

Latency

Oden Petersen

2026-09-30

About Me

- UNSW Maths/Compsci (graduated T1 2025)
- Data Engineering @ Arnott's Biscuits (2021)
- CPMSoc Maths Director 2021
- Data Scientist @ Daisee (2021-2022)
- Started QuantSoc 2022
- Quant Intern @ Autumn Compass (2022)
- Quant Intern @ Citadel Securities (Summer 2023)
- Trader @ Vivcourt (Summer 2024, Feb-Jul 2026)

Point of This Talk



Outline

- 1 Trading
- 2 Time
- 3 Distance
 - WAN
 - Colocation
- 4 Networking
- 5 Execution
- 6 Matching Engine
- 7 Market Data
- 8 Wire-to-wire
- 9 Hardware Design
- 10 Software Design
- 11 Programming
- 12 GPU Inference
- 13 Bibliography

Trading

“All economic activity is carried out through time. Every individual economic process occupies a certain time, and all linkages between economic processes necessarily involve longer or shorter periods of time.” Friedrich Hayek

Economic Considerations

Electronic Trading

Latency

```
> date +%s%N
1788413610858617734
```

Epoch timestamp: 1788413610 858 617 734
epoch seconds millis μ s nanos

```
> ping -c 5 192.168.0.109
PING 192.168.0.109 (192.168.0.109): 56 data bytes
64 bytes from 192.168.0.109: icmp_seq=0 ttl=64 time=3.763 ms
64 bytes from 192.168.0.109: icmp_seq=1 ttl=64 time=5.376 ms
64 bytes from 192.168.0.109: icmp_seq=2 ttl=64 time=7.845 ms
64 bytes from 192.168.0.109: icmp_seq=3 ttl=64 time=43.958 ms
64 bytes from 192.168.0.109: icmp_seq=4 ttl=64 time=4.174 ms

--- 192.168.0.109 ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 3.763/13.023/43.958/15.533 ms
```

Time is Money

Time



"Time exists in order that everything doesn't happen all at once." Ray Cummings



"By a clock we understand anything characterized by a phenomenon passing periodically through identical phases so that ... all that happens in a given period is identical with all that happens in an arbitrary period." Albert Einstein

Clocks

Quartz powered by a DC signal vibrates due to the piezoelectric effect at a reliable **reference frequency**.

A **voltage-controlled oscillator** has a frequency dependent on the input voltage.

A **phase detector** can produce a signal whose strength depends on how fast or slow the VCO is running compared to the reference.

If the VCO runs at a multiple of the reference frequency and is kept in a **phase-locked loop** with the reference, we can achieve oscillation at some multiple of the reference frequency. This can be chained a number of times.

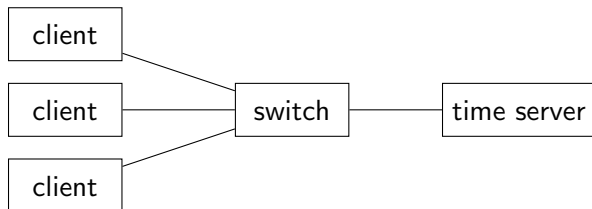
Heat, manufacturing differences, crystal aging, voltage fluctuations, etc., can cause two clocks $T_1(t)$ and $T_2(t)$ to experience **clock drift**, whereby $\theta_{1 \rightarrow 2} = T_2(t) - T_1(t)$ changes over time.

It is possible to observe negative apparent latency when naively comparing timestamps from poorly synced clocks.

Clock Drift

Synchronisation

For synchronisation within a network we will typically have a time server with a **master clock**. This could be set just once or synced with some external source like GPS.



Each piece of hardware can be equipped with its own clock $T_*(t)$. By sending messages across the network, recording timestamps at each of the clocks, and adjusting for latency, we can estimate the clock drift. This allows us to adjust the client clock speed (**slewing**) such that $\theta_{c \rightarrow m}$ approaches zero.

Network Time Protocol

Using the **network time protocol** involves the client making requests to the time server that are labeled with four receive/transmit timestamps $T_c(t_1)$, $T_m(t_2)$, $T_m(t_3)$, $T_c(t_4)$.

Suppose we have symmetric latency $T_m(t_2) - T_m(t_1) = T_m(t_4) - T_m(t_3)$.

It follows that $T_m(t_2) - (T_c(t_1) + \theta_{c \rightarrow m}) = (T_c(t_4) + \theta_{c \rightarrow m}) - T_m(t_3)$.

Then

$$\begin{aligned}\theta_{c \rightarrow m} &= \frac{1}{2} \left(\underbrace{(T_c(t_4) - T_m(t_3))}_{\text{Receive}} - \underbrace{(T_m(t_2) - T_c(t_1))}_{\text{Transmit}} \right) \\ &= \frac{1}{2} \left(\underbrace{(T_c(t_4) - T_c(t_1))}_{\text{Round-trip time}} - \underbrace{(T_m(t_3) - T_m(t_2))}_{\text{Wire-to-wire}} \right)\end{aligned}$$

In the case where wire-to-wire latency is zero, this is also known as Einstein synchronisation.

Precision Time Protocol

Because the latency added by a switch is nondeterministic, we can achieve greater precision by controlling for its contribution.

Suppose we have

$T_c(t_1), T_s(t_2), T_s(t_3), T_m(t_4), T_m(t_5), T_s(t_6), T_s(t_7), T_c(t_8)$.

Then we have

$$\theta_{c \rightarrow s} = \frac{1}{2} ((T_c(t_8) - T_s(t_7)) - (T_s(t_2) - T_c(t_1))),$$

$$\theta_{s \rightarrow m} = \frac{1}{2} ((T_s(t_6) - T_m(t_5)) - (T_m(t_4) - T_s(t_3))),$$

$$\theta_{c \rightarrow m} = \theta_{c \rightarrow s} + \theta_{s \rightarrow m}.$$

We can either record a cumulative adjustment in the timing packets (transparent switch), or sync the switch to the master clock and then the client to the switch (boundary clock).

Distance

"Speed is the essence of war." Sun Tzu

"Remote fountains are of little help to nearby fires." Han Fei

Things that are further away usually take longer to get a reply from:

```
> ping -c 3 192.168.0.109
PING 192.168.0.109 (192.168.0.109): 56 data bytes
64 bytes from 192.168.0.109: icmp_seq=0 ttl=64 time=2.593 ms
64 bytes from 192.168.0.109: icmp_seq=1 ttl=64 time=12.022 ms
64 bytes from 192.168.0.109: icmp_seq=2 ttl=64 time=53.092 ms

--- 192.168.0.109 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 2.593/22.569/53.092/21.924 ms
```

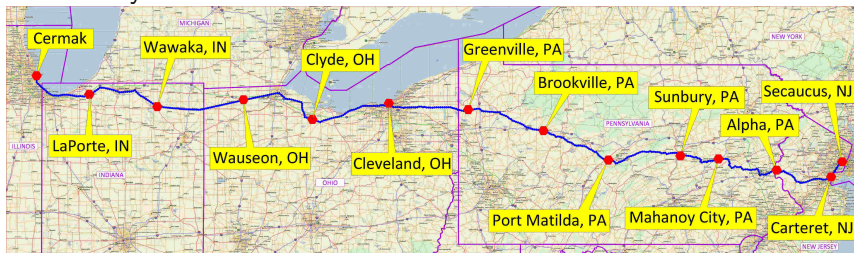
```
> ping -c 3 46.17.63.250
PING 46.17.63.250 (46.17.63.250): 56 data bytes
64 bytes from 46.17.63.250: icmp_seq=0 ttl=30 time=403.366 ms
64 bytes from 46.17.63.250: icmp_seq=1 ttl=30 time=338.468 ms
64 bytes from 46.17.63.250: icmp_seq=2 ttl=30 time=336.076 ms

--- 46.17.63.250 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 336.076/359.303/403.366/31.172 ms
```

Private Wide Area Network Links

1850: Paul Reuter establishes a carrier pigeon service between financial hubs in Germany and Belgium

2010: Spread Networks lays an optic fiber network between Chicago and New Jersey



2012: McKay Brothers installs an overground microwave tower relay

2017: Go West consortium consolidates fiber optic, microwave, and undersea cables between Chicago and Tokyo

Supposedly, shortwave radio can be bounced off the ionosphere to traverse oceans faster than the speed of light in glass.

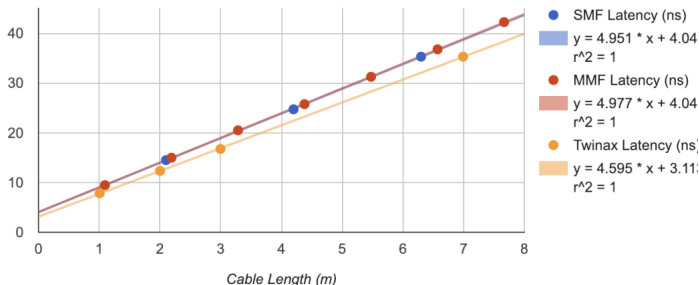
Physical Media

Latency is either along the cable or on one of the ends.

The longer the distance, the more we care about effects that scale linearly with distance.

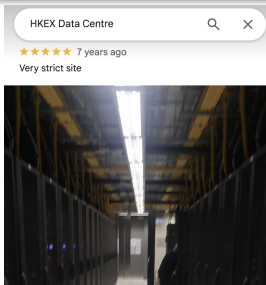
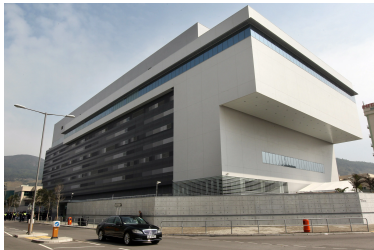
For private WAN links, we care mostly about the **propagation delay** of the medium, and the overhead required to remedy **attenuation**.

Cable Latencies



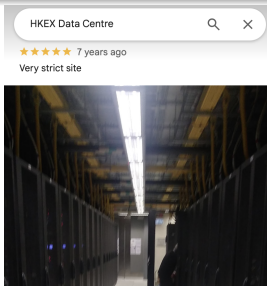
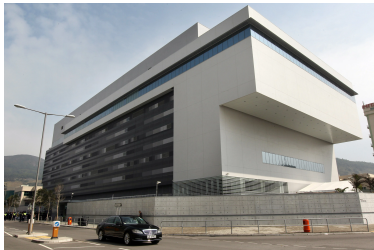
Direct Market Access

Remote connection < Proximity hosting < Primary colocation



Direct Market Access

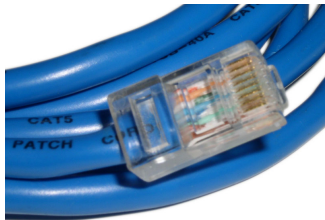
Remote connection < Proximity hosting < Primary colocation



Jan 19 (Reuters)

China's securities regulator has asked brokers to remove client-dedicated servers from data centers in local exchanges, a move that will deprive rapid-trading "flash boys" of an edge against other investors, people with knowledge of the matter said. High-frequency traders in China have long used servers situated at data centers run by futures and stock exchanges and owned by brokers to execute trades as the physical adjacency shaves off milliseconds or even microseconds.

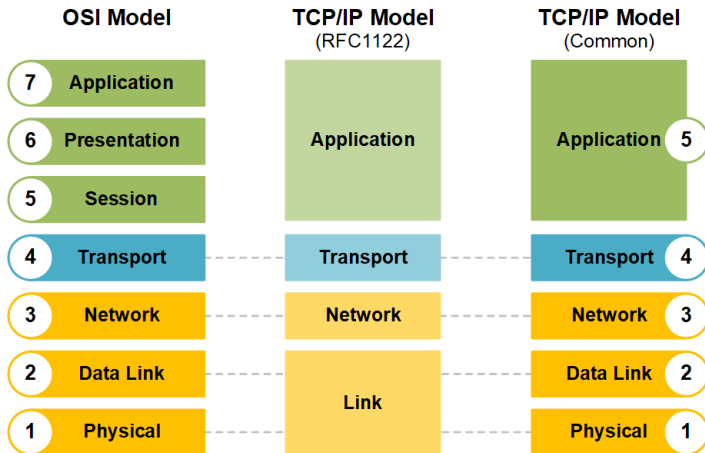
Networking



"This new form of communication could have some utility." Guglielmo Marconi



Networking Stack



- 1969: NPL network becomes operational, governed by a central 'routing' computer rather than an L1 crosspoint
- 1977: Patent granted for Ethernet, introducing CSMA/CD
- 1986: HP announces StarLAN, one of the first Ethernet hubs.
- 1986: DEC introduces the LANBridge 100 with a software-level MAC table and selective casting.
- 1989: Kalpana introduces EtherSwitch with a hardware-level MAC table, allowing half-duplex cut-through switching
- 1995: Cisco releases the Catalyst 5000 switch, supporting multicast switching
- 1999: Cisco releases the Catalyst 6000 switch, supporting Pulse Amplitude Modulation

L2: Ethernet

802.3 Ethernet packet and frame structure

Layer	Preamble	Start frame delimiter (SFD)	MAC destination	MAC source	802.1Q tag (optional)	Ethertype (Ethernet II) or length (IEEE 802.3)	Payload	Frame check sequence (32-bit CRC)	Interpacket gap (IPG)
Length (octets)	7	1	6	6	(4)	2	42–1500 ^[c]	4	12
Layer 2 Ethernet frame	(not part of the frame)		← 64–1522 octets →						(not part of the frame)
Layer 1 Ethernet packet & IPG	← 72–1530 octets →								← 12 octets →

L3: IP(v4)

IPv4 header format

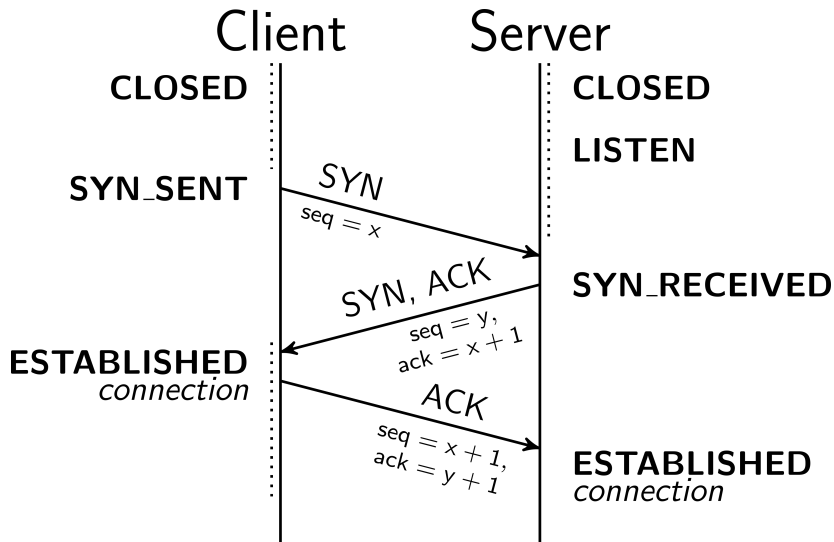
Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Version (4)				IHL				DSCP						ECN		Total Length															
4	32	Identification																Flags			Fragment Offset												
8	64	Time to Live								Protocol								Header Checksum															
12	96	Source address																															
16	128	Destination address																															
20	160	(Options) (if IHL > 5)																															
⋮	⋮																																
56	448																																

L4: UDP

UDP header format^[7]

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Length																Checksum															
8	64	Data																															
12	96																																
⋮	⋮																																

L4: TCP



TCP Segment

TCP header format^[20]

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set) ^[21]															
20	160	(Options) If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448	Data																															
60	480																																
64	512																																
⋮	⋮																																

Exchange Protocols

Network Timestamping

Timestamping of messages may be done at multiple points in the system (e.g. `timestamp_match` and `timestamp_publication`).

Timestamps from different clocks (e.g. from different machines) are not trivial to compare owing to clock drift and clock sync inaccuracies (e.g. caused by network asymmetry).

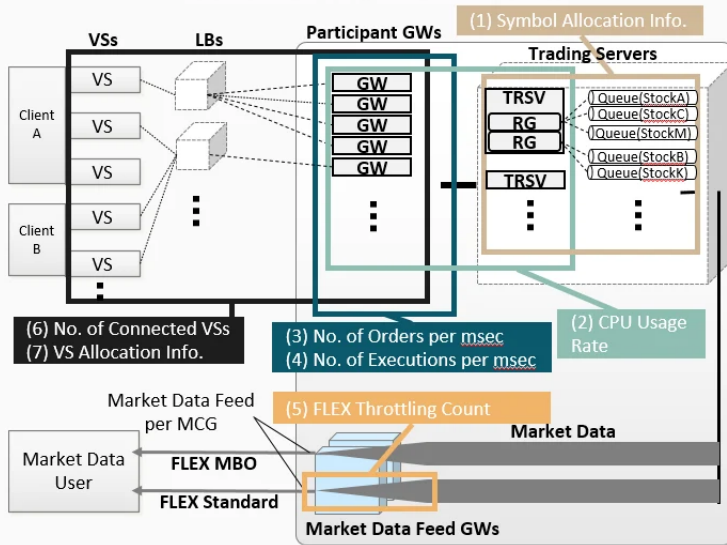
Usually the timestamp is calculated according to the first bit of the Ethernet header (start-of-frame). The last bit timestamp can be inferred using the bandwidth of the transmission.

Execution

*"We affirm that the world's magnificence has been enriched by a new beauty:
the beauty of speed." Filippo Marinetti*

Colocation Topology

Visualization of the Data Set



“What if you want to be a bit more unpleasant? How about you work out ... **which is the best gateway to connect to?** ... You can start sending your orders through that.

What about all these **other gateways**? What if you start stuffing orders through those other gateways that you don't care for matching?

... They're now **stuffed up with traffic** so other people can't compete.

Then you go through the gateway that you have set up which you know is fast, and you're going to send your orders through nice and quick.” Martin

Thompson, *Evolution of Financial Exchange Architecture*

“What if you want to be a bit more unpleasant? How about you work out ... **which is the best gateway to connect to?** ... You can start sending your orders through that.

What about all these **other gateways**? What if you start stuffing orders through those other gateways that you don't care for matching?

... They're now **stuffed up with traffic** so other people can't compete.

Then you go through the gateway that you have set up which you know is fast, and you're going to send your orders through nice and quick.” Martin

Thompson, *Evolution of Financial Exchange Architecture*

Load balancers help accomodate the large volume of data moving through the network, particularly during significant market events. Exchanges often impose limits on throughput (**‘transactions per second’**). Limits might be per-session, per-user, etc.

In order to ensure every EP can trade in a fair and orderly market, activities including the following ... are therefore prohibited:

- ① Deliberate fragmentation of Transmission Control Protocol (TCP) messages ('packet splitting');
- ② Deliberate retransmission of TCP or application layer messages;
- ③ Deliberate transmission of corrupt network packets;
- ④ Inappropriate/excessive use of non-trading messages such as heartbeat, connection management; and
- ⑤ Hold up or interfere with normal queuing and processing of network or application messages.

“The virtual channels on the X.25 for ordering were only 64kbps – very slow. I figured we could hack those similarly . . . and start filling out an order before we had one. This would save us milliseconds. You could turn the packet into something benign or invalid if you didn't have an order.”

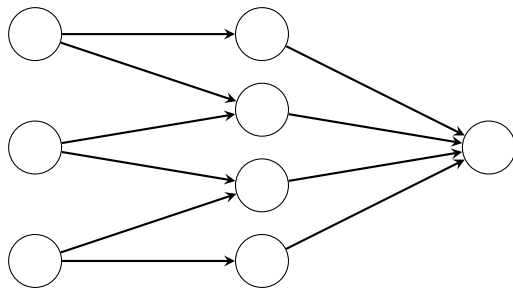
Queueing Models

Suppose all participants have independent latency with mean μ_i and a common standard deviation σ (also called **jitter**).

If we model latency as a negative Gumbel distribution, the probability of being fastest is conveniently proportional to $\exp\left(\frac{\pi}{\sqrt{6}} \cdot \frac{-\mu_i}{\sigma}\right)$.

If every participant sends n_i independent copies of their message, the probability of being fastest is proportional to $n_i \cdot \exp\left(\frac{\pi}{\sqrt{6}} \cdot \frac{-\mu_i}{\sigma}\right)$.

Of course, the latencies are very often not independent.



Lottery Game

An online lottery website is offering a \$1m jackpot. Unfortunately, because they vibecoded the website, nobody is able to buy any tickets. Even worse, n users have each been given one free ticket. Each number i has some probability p_i of winning, in which case the jackpot will be split amongst all players who chose i . If nobody wins, the jackpot will not pay out. How should the players choose their numbers?

Lottery Game Solution

If a player chooses each number i with probability π_i , and other players with probability π'_i , the player's expected payoff (in millions of dollars) is

$$\begin{aligned} & \pi_i \cdot p_i \cdot \left(\sum_{k=0}^{n-1} \frac{1}{1+k} \binom{n-1}{k} \pi_i'^k (1 - \pi_i')^{n-1-k} \right) \\ &= \pi_i \cdot p_i \cdot \left(\sum_{k=0}^{n-1} \left(\int_0^1 x^k dx \right) \binom{n-1}{k} (\pi_i')^k (1 - \pi_i')^{n-1-k} \right) \\ &= \pi_i \cdot p_i \cdot \int_0^1 \left(\sum_{k=0}^{n-1} \binom{n-1}{k} (\pi_i' \cdot x)^k (1 - \pi_i')^{n-1-k} \right) dx \\ &= \pi_i \cdot p_i \cdot \int_0^1 (1 - \pi_i' + \pi_i' \cdot x)^{n-1} dx = \pi_i \cdot p_i \cdot \frac{1 - (1 - \pi_i')^n}{n\pi_i'}. \end{aligned}$$

For a symmetric equilibrium, we require that $p_i \cdot \frac{1 - (1 - \pi_i')^n}{n\pi_i'}$ is the same for all i . If $(1 - \pi_i')^n$ is negligible this reduces to $\pi_i' \propto p_i$.

Matching Engine

“The most amazing achievement of the computer software industry is its continuing cancellation of the steady and staggering gains made by the computer hardware industry.” Henry Peteroski

Matching Engine Responsibilities

- Handle trading session initialisation and publication of reference data
- Capture messages from the network and assign an order in which they are to be processed
- Parse and validate messages, including risk checks
- Apply valid changes to **alter the order book state**
- **Produce trades** when appropriate conditions are met
- Log market events for clearing & regulatory review
- Publish market events & order status back to participants
- Handle heartbeats, disconnects, and recovery

Matching Engine Requirements

The main goal of matching engine design is **reliability**. For the exchange, latency is a secondary consideration.

- Throughput under worst-case load
 - Load balancing
 - Multi-threaded validation/parsing
 - Sharding/partitioning
 - Caching
- Continuous availability
 - Live backup server
 - Heartbeat messages
- State machine determinism
 - Single-threaded transaction processing
- Fairness

“The transition to TCP was a little messy . . . Sometimes the feed imbalances would have severe delays, with data lagging behind tens of minutes. Once the imbalance cleared, the engines would automatically go back to trading.”

Order Book Representation

Market Data

*"The race is not to the swift, nor the battle to the strong . . . but time and chance
happeneth to them all." Ecclesiastes*

*"The race is not always to the swift, nor the battle to the strong – but that's the
way to bet." Anon.*

Market Data Types

Public Feed

- Trades
- Depth
 - L1: Best Bid/Offer (BBO)
 - L2: Level sizes
 - L3: Market-by-order
 - Incremental
 - Snapshot

Private Feed

- Fills
- Acknowledgement of order creation/deletion

Often we will get the same information in more than one way.

Trade Message Example (SGX OMnet)

```
struct BD70 { // Trade Ticker VIB (Variable Information Broadcast)
    struct broadcast_hdr {
        struct broadcast_type {
            CHAR central_module_c; CHAR server_type_c;
            UINT16_T transaction_number_n // Transaction Type Number
        }
        UINT16_T items_n // Items
        UINT16_T size_n // Size
    }
    struct sub_item_hdr {
        UINT16_T named_struct_n // Named Struct, Number
        UINT16_T size_n // Size
    }
    struct basic_trade_ticker { // Named struct no: 34401
        struct series { // Named struct no: 50000
            UINT8_T country_c; UINT8_T market_c; UINT8_T instrument_group_c;
            UINT8_T modifier_c; UINT16_T commodity_n; UINT16_T expiration_date_n;
            INT32_T strike_price_i
        }
        struct timestamp_match { UINT32_T tv_sec; INT32_T tv_nsec }
        struct time_of_publication { UINT32_T tv_sec; INT32_T tv_nsec; }
        UINT64_T execution_event_nbr_u // Execution number %sequence number
        UINT32_T match_group_nbr_u // Match group number, group inside an execution
        INT64_T deal_quantity_i // Quantity, Deal
        INT32_T deal_price_i // Price, Deal
        UINT16_T segment_number_n // Segment Number
        UINT8_T aggressive // Bid or Ask ; Of type: BID_OR_ASK_C
        CHAR filler_1_s // Filler
    }
}
```

Private Fill Message Example

```
struct B05 { // Firm Order Book VIB
    struct broadcast_hdr { ... }
    struct sub_item_hdr { UINT16_T named_struct_n; UINT16_T size_n }
    struct trade_report_1_trans { // Named struct no: 34021
        struct transaction_type {
            CHAR central_module_c; CHAR server_type_c;
            UINT16_T transaction_number_n // Transaction Type Number
        }
        struct series { ... }
        struct order_var {
            INT64_T mp_quantity_i; INT32_T premium_i; UINT32_T block_n;
            UINT16_T time_validity_n; UINT16_T exch_order_type_n;
            UINT16_T trigger_order_time_validity_n; char[16] ex_client_s;
            char[15] customer_info_s; UINT8_T open_close_req_c;
            UINT8_T bid_or_ask_c; UINT8_T ext_t_state_c;
            UINT8_T order_type_c; UINT8_T stop_condition_c; char[2] filler_2_s;
            %Metadata, look for patterns. https://security.stackexchange.com/questions/216369/is-exposing
        }
        struct party {char[2] country_id_s; char[5] ex_customer_s; CHAR filler_1_s}
        CHAR[32] exchange_info_s // Exchange, Information
        struct give_up_member {
            char[2] country_id_s; char[5] ex_customer_s; CHAR filler_1_s;
        }
        char[8] settlement_date_s // Date, Settlement
        char[8] time_of_agreement_date_s // Time of agreement, date part
        char[6] time_of_agreement_time_s // Time of agreement, time part
        UINT8_T deferred_publication_c // Deferred Publication
        CHAR filler_1_s // Filler
    }
}
```

During a burst of activity, the ME may need to do post-processing of data prior to sending.

More complex transactions may take longer to be processed by the matching engine, or to traverse through the network.

Larger messages may be **packetised** and packets may be **fragmented** (according to the L2/L3 Maximum Transmission Unit).

Public Market Data

This is ideally **multicast** (or broadcast) to participants depending what kinds of feeds and products they choose to subscribe to. For fairness and latency reasons, public market data is ideally emitted only once by the matching engine NIC, then sent to a multicast switch that transmits to all relevant recipients simultaneously. Once received, packets can be further filtered or triaged by participants.

Public Market Data

“At one later site in the new KRX Annexe, we put in market data and it was the worst. . . . We asked for this market data to be removed. A month or two later we paid for it to be reinstalled . . . and now it was one of the best. Such were the vagaries of life in Korea. I suspect some new kind of semi-permanent **round robin** positioning somewhere fixed things. There was little rhyme nor reason for many things.”

This is typically **unicast**, and the matching engine may need to decide on some reasonable order in which to send updates to participants. Depending on how this is achieved, there might be ways to gain priority in the distribution of private feed messages. Because only certain events emit private feed information, different styles of trading will be more or less likely to result in a private update about latency-sensitive events.

Wire-to-wire

"Time is the inexorable process of doing computation." Stephen Wolfram

Network Interface Cards

A copper pair passing into an exchange participant rack will interface with one or more **network interface cards** (NICs).

Receive (RX) Responsibilities

- Strip ethernet preamble, filter by destination MAC address, verify/strip FCS
- Buffer incoming data & stream to packet buffers in RAM for Direct Memory Access (DMA)
- Decide when and whether to send interrupt to CPU
- Receive Side Scaling (RSS)

Transmit (TX) Responsibilities

- Add Ethernet preamble (& FCS)
- Stream data from RAM using DMA
- Hold frames in TX buffer prior to serialisation

The wire-to-wire latency of any task will be determined by the **hotpath** of that task within our system.



An onboard **trigger cache** lets us store parameters to determine how to respond to market data.

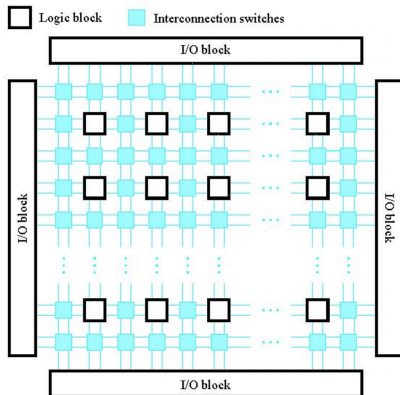
Precomputing these before the latency-critical moment reduces hotpath latency.

However, because of the large volume of market data and limited space in the trigger cache, **tick-to-config** latency/throughput are still important considerations.

Hardware Design

“All great events hang by a hair.” Napoleon

Field Programmable Gate Arrays



One example is the X3522 accelerator card.
While primarily useful onboard a NIC, FPGAs also have applications in other low-latency/high-throughput situations.

Interconnection

Software Design

“You don’t need to be an engineer to be a racing driver, but you do need Mechanical Sympathy.” Jackie Stewart

Memory

Machine Instructions

Multitasking

Kernel Bypass

Programming

"There is no science apart from the general.

It may even be said that the very object of the exact sciences is to spare us these direct verifications." Henri Poincaré

GPU Inference

Bibliography



Classics & Mentioned

